

Advanced Deep Learning

Theory Lab 0

Christos Palantzidis

Theoretical Tasks

Task 0.1 Convolution

Given image matrix I and kernel k:

Image I

1	1	1	1
1	1	2	1
1	-3	-4	1
1	1	1	1

Kernel k

0	1	0
1	2	1
0	1	0

Because the task asks for same convolution with stride 1, zero-padding of size 1 is added around the image.

Padded image

0	0	0	0	0	0
0	1	1	1	1	0
0	1	1	2	1	0
0	1	-3	-4	1	0
0	1	1	1	1	0
0	0	0	0	0	0

First kernel position:

Patch 1

0	0	0
0	1	1
0	1	1

$$O(1,1) = 0*0 + 0*1 + 0*0 + 0*1 + 1*2 + 1*1 + 0*0 + 1*1 + 1*0 = 4$$

Second kernel position (one step to the right):

Patch 2

0	0	0
1	1	1
1	1	2

$$O(1,2) = 0*0 + 0*1 + 0*0 + 1*1 + 1*2 + 1*1 + 1*0 + 1*1 + 2*0 = 5$$

Continuing in the same way for all positions gives the final output:

Output

4	5	6	4
5	2	3	5
0	-7	-2	0
4	1	0	4

Final result:

[[4, 5, 6, 4], [5, 2, 3, 5], [0, -7, -2, 0], [4, 1, 0, 4]]

Task 0.2 Non Linearity

In convolutional neural networks, convolutional layers are usually followed by a non-linear activation function. A common choice is ReLU, which is defined as:

$$\text{ReLU}(x) = \max(0, x)$$

This means that all negative values are replaced by 0, while positive values remain unchanged.

Using the output from Task 0.1:

4	5	6	4
5	2	3	5
0	-7	-2	0
4	1	0	4

After applying **ReLU** element-wise, the output becomes:

4	5	6	4
5	2	3	5
0	0	0	0
4	1	0	4

Therefore, the result after the non-linearity is:

4	5	6	4
5	2	3	5
0	0	0	0
4	1	0	4

Task 0.3 Max Pooling

Max pooling divides the input into smaller regions and keeps only the maximum value from each region.

Using the result from Task 0.2:

[4 5 6 4]
 [5 2 3 5]
 [0 0 0 0]
 [4 1 0 4]

I apply valid max pooling with a filter size of 2×2 . This divides the input into the following four regions:

$\begin{bmatrix} 4 & 5 \\ 5 & 2 \end{bmatrix}$ max = 5	$\begin{bmatrix} 6 & 4 \\ 3 & 5 \end{bmatrix}$ max = 6
$\begin{bmatrix} 0 & 0 \\ 4 & 1 \end{bmatrix}$ max = 4	$\begin{bmatrix} 0 & 0 \\ 0 & 4 \end{bmatrix}$ max = 4

Therefore, the output after max pooling is:

$$\begin{bmatrix} 5 & 6 \\ 4 & 4 \end{bmatrix}$$

Task 0.4 Flattening

Flattening converts a matrix into a one-dimensional vector by placing all rows one after another. Using the result from Task 0.3:

$$\begin{bmatrix} 5 & 6 \\ 4 & 4 \end{bmatrix}$$

the flattened vector is:

$$[5, 6, 4, 4]$$

Therefore, the output after flattening is **[5, 6, 4, 4]**.

Task 0.5 Fully Connected Layer

After flattening, the feature vector from the previous task is:

$$x = [5, 6, 4, 4]$$

A fully connected layer performs a matrix multiplication between the input vector and a weight matrix. Since the exercise does not provide a specific weight matrix, I use the following example matrix:

$$W = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

Then the output of the fully connected layer is:

$$z = xW$$

$$z = [5, 6, 4, 4] \cdot W$$

Calculating each component:

$$z_1 = 5 \cdot 1 + 6 \cdot 0 + 4 \cdot 1 + 4 \cdot 0 = 9$$

$$z_2 = 5 \cdot 0 + 6 \cdot 1 + 4 \cdot 1 + 4 \cdot 1 = 14$$

$$z_3 = 5 \cdot 1 + 6 \cdot 1 + 4 \cdot 0 + 4 \cdot 0 = 11$$

Therefore, the output of the fully connected layer is:

$$z = [9, 14, 11]$$

Task 0.6 SoftMax

The output from the previous task is:

$$z = [9, 14, 11]$$

The softmax function converts these raw outputs into probabilities:

$$\text{softmax}(z_i) = e^{(z_i)} / \sum_j e^{(z_j)}$$

Therefore:

$$\text{softmax}([9, 14, 11]) = [e^9/(e^9 + e^{14} + e^{11}), e^{14}/(e^9 + e^{14} + e^{11}), e^{11}/(e^9 + e^{14} + e^{11})]$$

Factoring out e^{14} , we get:

$$[e^{(-5)}/(e^{(-5)} + 1 + e^{(-3)}), 1/(e^{(-5)} + 1 + e^{(-3)}), e^{(-3)}/(e^{(-5)} + 1 + e^{(-3)})]$$

Using approximate values:

$$e^{(-5)} \approx 0.0067, \quad e^{(-3)} \approx 0.0498$$

we obtain:

$$\text{softmax}([9, 14, 11]) \approx [0.0063, 0.9465, 0.0471]$$

Since the highest probability is the second value, the predicted output class is the second class.

Task 0.7 Loss Functions

Given:

$$g = [0, 1, 0]$$

$$y = [0.25, 0.6, 0.15]$$

Cross-Entropy Loss

$$H(y, g) = -\sum_i g_i \log(y_i)$$

Since the ground truth vector is one-hot encoded, only the second term remains:

$$H(y, g) = -\log(0.6) \approx 0.5108$$

Mean Squared Error Loss

$$\text{MSE}(y, g) = (1/3) * [(0.25 - 0)^2 + (0.6 - 1)^2 + (0.15 - 0)^2]$$

$$= (1/3) * [0.0625 + 0.16 + 0.0225]$$

$$= 0.245 / 3 \approx 0.0817$$

Hinge Loss

The correct class is the second one, so $j = 2$ and $y_j = 0.6$.

$$\text{SVM}(y, j) = \sum_{(i \neq j)} \max(0, y_i - y_j + 1)$$

$$= \max(0, 0.25 - 0.6 + 1) + \max(0, 0.15 - 0.6 + 1)$$

$$= 0.65 + 0.55 = 1.20$$

Therefore:

- Cross-Entropy Loss = 0.5108
- Mean Squared Error Loss = 0.0817
- Hinge Loss = 1.20

Evaluation Metrics

Task 0.1 – Theoretical Foundations

1. Accuracy is not always a good evaluation metric, especially when the dataset is imbalanced. In that case, a model may achieve a high accuracy by mostly predicting the majority class, even if it performs poorly on the minority class. So the result can look good, but in reality the model may not be very useful.
2. What makes accuracy less desirable than Jaccard Index is that it includes all correct predictions, including true negatives. In a multi-class setting, this can hide the real quality of the model. Jaccard Index is often more informative because it focuses more directly on the overlap between prediction and ground truth.
3. Jaccard Index and F1-score are closely related, but they are not the same. Jaccard measures the overlap between predicted and true values, while F1-score measures the balance between precision and recall. Jaccard is usually more suitable for overlap-based tasks, such as segmentation, while F1-score is often used in classification problems. So the most suitable metric depends on the type of neural network task.

Task 0.2 Practice

Since this is a multi-class and multi-label scenario, I evaluate each class separately.

The classes are:

B = background, T = text, D = decoration, C = comment.

The ground truth and predictions are:

GT: B, T, B, B, TD, TD, TD, TD

P : B, B, TD, BD, BC, TC, T, TD

First, I count the class frequencies in the ground truth:

B appears 3 times, T appears 5 times, D appears 4 times, and C appears 0 times.

Then I compute the metrics class by class.

For class B:

GT positives are positions 1, 3, 4 and predicted positives are positions 1, 2, 4, 5.

So TP = 2, FP = 2, FN = 1.

Jaccard = $2 / (2 + 2 + 1) = 0.40$

Precision = $2 / (2 + 2) = 0.50$

Recall = $2 / (2 + 1) = 0.6667$

F1 = $2 \cdot 2 / (2 \cdot 2 + 2 + 1) = 0.5714$

For class T:

GT positives are positions 2, 5, 6, 7, 8 and predicted positives are positions 3, 6, 7, 8.

So TP = 3, FP = 1, FN = 2.

Jaccard = $3 / (3 + 1 + 2) = 0.50$

Precision = $3 / (3 + 1) = 0.75$
Recall = $3 / (3 + 2) = 0.60$
F1 = $2 \cdot 3 / (2 \cdot 3 + 1 + 2) = 0.6667$

For class D:

GT positives are positions 5, 6, 7, 8 and predicted positives are positions 3, 4, 8.

So TP = 1, FP = 2, FN = 3.

Jaccard = $1 / (1 + 2 + 3) = 0.1667$

Precision = $1 / (1 + 2) = 0.3333$

Recall = $1 / (1 + 3) = 0.25$

F1 = $2 \cdot 1 / (2 \cdot 1 + 2 + 3) = 0.2857$

For class C:

There are no positives in the ground truth, while the model predicts class C in positions 5 and 6.

So TP = 0, FP = 2, FN = 0.

Jaccard = 0

Precision = 0

Recall = 0

F1 = 0

Final per-class results:

B: frequency = 3, Jaccard = 0.4000, Precision = 0.5000, Recall = 0.6667, F1 = 0.5714

T: frequency = 5, Jaccard = 0.5000, Precision = 0.7500, Recall = 0.6000, F1 = 0.6667

D: frequency = 4, Jaccard = 0.1667, Precision = 0.3333, Recall = 0.2500, F1 = 0.2857

C: frequency = 0, Jaccard = 0, Precision = 0, Recall = 0, F1 = 0

Next, I compute the mean with class balance, which is the simple average over the 4 classes:

Mean Jaccard = $(0.4000 + 0.5000 + 0.1667 + 0) / 4 = 0.2667$

Mean Precision = $(0.5000 + 0.7500 + 0.3333 + 0) / 4 = 0.3958$

Mean Recall = $(0.6667 + 0.6000 + 0.2500 + 0) / 4 = 0.3792$

Mean F1 = $(0.5714 + 0.6667 + 0.2857 + 0) / 4 = 0.3810$

Finally, I compute the weighted mean using the class frequencies.

The normalized class frequencies are:

B = $3/12 = 0.25$, T = $5/12 = 0.4167$, D = $4/12 = 0.3333$, C = $0/12 = 0$.

Weighted Jaccard = $0.25 \cdot 0.4000 + 0.4167 \cdot 0.5000 + 0.3333 \cdot 0.1667 + 0 \cdot 0 = 0.3639$

Weighted Precision = $0.25 \cdot 0.5000 + 0.4167 \cdot 0.7500 + 0.3333 \cdot 0.3333 + 0 \cdot 0 = 0.5486$

Weighted Recall = $0.25 \cdot 0.6667 + 0.4167 \cdot 0.6000 + 0.3333 \cdot 0.2500 + 0 \cdot 0 = 0.5000$

Weighted F1 = $0.25 \cdot 0.5714 + 0.4167 \cdot 0.6667 + 0.3333 \cdot 0.2857 + 0 \cdot 0 = 0.5159$

Final results:

Class-balanced mean: Jaccard = 0.2667, Precision = 0.3958, Recall = 0.3792, F1 = 0.3810

Frequency-weighted mean: Jaccard = 0.3639, Precision = 0.5486, Recall = 0.5000, F1 = 0.5159